Foodtech AI Recipe Agent

프로젝트 중간 보고서

분석·설계 진행과 구현 준비 현황

보고 기준일 2026년 10월 24일

프로젝트 수행자 김범수

문서 상태 중간 점검본

목차

1 보고 범위와 현재 결론 3

2 사업 요구와 개인 프로젝트의 범위 4

3 요구사항과 설계 산출물의 진척 5

4 데이터 수집과 Mock 검증 설계 6

5 조리 시나리오와 AI 권고 8

6 시스템 구조와 운영 안전장치 9

7 현재 검증 결과와 해석 범위 11

8 남은 결정과 다음 단계 12

참고문헌 14

1 보고 범위와 현재 결론

Foodtech AI Recipe Agent는 조리로봇과 주방장비에서 읽은 데이터를 같은 의미로 표준화하고, 메뉴·매장·조리 결과와 연결하여 운영자가 레시피와 권장 조리조건을 판단하도록 돕는 의사결정 지원 플랫폼이다. 현재까지는 분석 문서와 설계 초안이 마련되었다. 제품 서버, 독립 Mock 제어 프로그램, 고객·관리자 웹이 통합 동작한다는 증거는 아직 없다.[1][2]

서로 다른 장비가 같은 온도를 보내더라도 그것이 챔버 측정값인지, 솥 표면 측정값인지, 가열 설정값인지에 따라 조리 판단에서 쓰는 위치가 달라진다. 프로젝트는 이 차이를 데이터 항목의 의미와 출처까지 기록한 뒤 조리작업에 연결하도록 설계한다. 보고의 관심도 문서가 몇 개 만들어졌는가보다, 이 연결을 실제 서비스에서 재현할 준비가 어디까지 되었는가에 있다.[2][3]

보고일은 2026년 8월 31일 시작, 12월 7일 종료로 계획한 15주 일정의 8주차에 해당한다. 시작 주를 1주차로 세는 프로젝트 규칙을 적용한 표기이며, 이 주차가 기능 완료율을 뜻하지는 않는다.[2]

구분	현재 확인된 상태	해석
분석	요구사항 정의서·명세서, 데이터 수집 정의서의 최신 제안본 존재	요구와 수집 책임을 기록했으나 고객 수락이나 제품 시험의 완료가 아님
설계	아키텍처·개발표준·화면·UML·ERD·테이블·코드북 초안 존재	물리 키·구현 계약과 일부 임계치는 후속 확정 대상
부분 검증	Python·메모리 SQLite로 일부 불변 조건 및 후보 규칙 점검	실제 장비·AWS·서비스 성능 검증이 아님
제품 구현	검토한 원 프로젝트에서 제품 서비스·Mock 실행 코드의 완료 증거 미확인	문서 사이트의 동작을 제품 기능 구현으로 계산하지 않음

표 1 현재 진행 상태와 판정 범위. 문서의 존재, 설계의 승인 방향, 구현·시험 결과를 별도로 읽어야 한다.

이후 개발은 설계의 미결정 계약을 기능별로 좁히고, Mock에서 생성한 사건을 실제 수집·표준화·판정·권고 흐름에 통과시킨 뒤, 로컬과 클라우드에서 서로 다른 검증을 수행하는 순서가 적절하다. 현재 정해진 수치 목표나 안전 판정 예시를 실측 성과로 읽지 않는 것이 중간 보고의 핵심 경계다.[3]

현재 확인 가능한 성과는 사업 요구를 개인 프로젝트의 범위로 해석하고, 각 기능이 받아야 할 입력과 남겨야 할 기록을 분석·설계 문서에 연결한 것이다. 로컬의 작은 계약 시험은 이 방향이 일부 예시에서 성립할 수 있음을 보여주지만 장비 연결, 다중 사용자의 경쟁 처리, 클라우드 권한과 처리량은 확인하지 않았다. 따라서 이후의 진척은 문서 작성량이 아니라 실행 가능한 종단간 흐름과 그 시험 증적으로 판단해야 한다.[3][4]

2 사업 요구와 개인 프로젝트의 범위

출발점인 공개 과업지시서는 장비별로 다른 데이터 구조를 통합하고, 조리조건과 운영환경을 분석하여 품질 편차를 줄이는 플랫폼을 요구한다. 원 사업은 테스트베드 장비 연계와 현장 적용도 포함한다. 원문 자체가 플랫폼을 장비 직접 제어 시스템이 아닌 데이터 수집·분석과 레시피 권고 시스템으로 규정한다.[5]

조리 품질이 일정하지 않은 문제를 다루려면 결과 평가만 모아서는 부족하다. 어떤 장비에서 어떤 레시피 버전으로 조리했고, 투입·가열·인양 때 실제 측정값이 어떠했는지 알아야 매장과 배치 간 차이를 해석할 수 있다. 원 사업이 데이터 통합을 먼저 요구한 이유를 개인 프로젝트에서는 이 추적 가능한 조리기록의 필요성으로 해석했다. 장비별 명칭과 단위를 그대로 저장하면 같은 조건의 비교도 불가능하므로 표준화가 선행한다.[5][1]

개인 프로젝트는 실물 장비와 제조사별 비공개 연계 계약을 확보하지 못한 상태를 전제로 한다. 따라서 실제 제조사 호환 구현 대신 가상 제조사 데이터와 Mock 송신기를 만들고, 연계는 읽기 전용으로 제한한다. AI 제안은 운영자의 검토·승인을 거치며 장비에 설정값을 자동 전송하지 않는다. 이 전환은 안전·계약·재현 가능한 시험 범위를 분리하기 위한 선택이다.[1][2]

Mock은 원 사업의 현장 실증을 대체하지 않는다. 공개 정보와 명시적인 가상 규칙으로 필드 누락, 단위 차이, 통신 오류를 반복 재현해 수집 소프트웨어를 시험한다. 실제 제조사에서 확인되지 않은 필드·코드·주기는 가상값으로 표시하며 제조사 규격이나 현장 성능으로 보고하지 않는다.[6]

원 사업에서 요구한 일	개인 프로젝트의 적용	현재 상태
테스트베드 장비와 현장 연계	공개자료에서 도출한 합성 Mock·세 가상 제조사 fixture로 대체	설계 결정, 실물 호환성 미검증
물리 서버와 현장 운영	논리 배포 구성과 로컬 개발·클라우드 시험·운영 분리	설계 초안, AWS 적합성 미검증
레시피 생성·고도화	권고 후보, 사람의 승인, 결과와 평가 이력으로 범위 재구성	요구·화면·서비스 경계 설계

표 2 사업 요구의 개인 프로젝트 전환. ‘대체’는 원 사업의 장비·현장 성과를 달성했다는 뜻이 아니다.

프로젝트가 다루는 사람은 현장 운영자, 매장 관리자, 시스템 관리자, 개발·시험 담당자다. 고객 웹은 질문과 작업·레시피·평가를 제공하고, 관리자 웹은 장비 프로파일과 수집·권한·감사를 다룬다. Mock 제어 프로그램은 고객 화면이나 관리자 화면의 하위 메뉴가 아니라 별도 시험 도구로 설계했다.[7]

3 요구사항과 설계 산출물의 진척

요구사항 정의서 RD(Requirements Definition)는 왜 이 시스템이 필요한지와 포함·제외 범위를 정한다. 요구사항 명세서 SRS(Software Requirements Specification)는 입력, 처리, 예외, 권한과 수용 조건을 기능별로 구체화한다. 데이터 수집 정의서 DCD(Data Collection Definition)는 무엇을 어디서, 어떤 방식과 주기로 수집할지 맡는다. 세 문서는 모두 ‘제안 단계’이며 최신 공개본은 각각 RD v0.7, SRS v0.7, DCD v0.6이다.[1][2][6]

설계 문서는 같은 책임을 반복하지 않는다. 시스템 아키텍처 정의서 SAD(System Architecture Definition)는 프로그램과 신뢰 경계, 개발표준정의서는 코드·데이터·시험 규칙, 정보구조·화면 정의서 IA/Storyboard는 사용자 행동과 화면 상태, UML 설계서는 호출 순서와 클래스 책임을 설명한다. ERD, 테이블 정의서와 코드북은 논리 관계, 물리 저장과 데이터 의미를 구분한다. 이들 역시 구현 전 제안본이다.[3][7][8][9][10][11]

문서 묶음	최신 공개 버전	현재 독자가 확인할 것
분석	RD 0.7 · SRS 0.7 · DCD 0.6	범위, 기능·검증 조건, 수집 소스와 책임
구조·개발	SAD 0.3 · 개발표준 0.4 · UML 0.3	논리 모듈, 의존 방향, 정상·예외 흐름
화면	IA/Storyboard 0.4	고객·관리자·Mock UI의 분리와 화면별 상태
데이터	ERD 0.2 · 테이블 0.2 · 코드북 0.2	참조 관계, 접근 패턴, 코드·단위의 의미

표 3 현재 문서의 역할. 버전은 보고일에 원 프로젝트와 Resources 자료실에서 확인한 공개 최신본이다.

문서가 구체적이어도 실제 저장소의 파티션 키, 보조 인덱스, 조건부 쓰기, 서비스 경계의 적합성이 자동으로 증명되지는 않는다. 이전 로컬 검증 기록은 작성 당시의 더 이른 문서 버전을 대상으로 기술한 부분이 있어, 최신 문서 전체를 통과한 검증 결과로 확대하지 않는다.[3][4]

한 기능을 개발할 때는 RD의 범위, SRS의 정상·예외·권한 조건, DCD의 원천 데이터 정의를 먼저 서로 대조해야 한다. 예를 들어 '온도 수집'은 장비에서 숫자를 받는 것으로 끝나지 않는다. 원천 단위와 측정 위치를 확인하고, 변환 규칙과 품질 상태를 남긴 뒤, 어떤 조리작업과 레시피 버전의 판정에 쓰였는지까지 이어져야 한다. SAD와 UML은 이 처리의 소유 모듈과 호출 순서를, ERD·테이블·코드북은 연결할 식별자와 값의 뜻을 제시한다. 이 추적 관계가 실행 코드에서도 유지되는지는 후속 시험 대상이다.[1][2][6][3][9][10][11]

4 데이터 수집과 Mock 검증 설계

실제 장비가 없다면 정상 데이터만 예쁘게 만드는 시험은 충분하지 않다. 이 프로젝트의 Mock은 프로토콜·단위·필드명·상태 코드·누락·중복·순번 역전과 연결 단절을 재현해, 수집 이후의 표준화와 품질 판정이 원천 차이를 견딜 수 있는지 검증하도록 설계됐다. Mock은 실제 제조사 API나 통신규격과 호환된다고 주장하지 않는다.[6][3]

프로파일 계열은 REST, MQTT, OPC UA라는 세 연계 방식과 장비 유형·데이터 유형을 조합한 21개다. 각 계열에 vendor-a/b/c의 가상 제조사 표현을 적용하면 설계상 최소 63개 인스턴스가 된다(21계열 × 3 fixture). 이 수는 생성

완료 개수나 실제 처리량이 아니라 시험 범위의 최소 구성이다. vendor-a는 camelCase·중첩·섭씨, vendor-b는 snake_case·평탄화·화씨, vendor-c는 PascalCase·혼합 구조 등으로 차이를 낸다.[6]

이 분리는 세 가지 질문을 각각 시험하기 위한 것이다. 장비 유형은 솥 표면온도와 오븐 챔버온도처럼 무엇을 표현하는지를, 프로파일은 REST·MQTT·OPC UA와 JSON·XML·CSV 등 어떤 경로와 형식으로 전달하는지를, fixture는 같은 뜻을 필드명·시각·단위·상태 코드로 어떻게 적는지를 담당한다. vendor-b의 화씨·Unix 밀리초·정수 상태 코드는 vendor-a의 섭씨·UTC 시각·문자열 상태와 의미가 같을 수 있으나 변환 규칙 없이는 동일 값으로 취급하지 않는다.[6]

생성 항목의 출처도 공개자료에서 직접 확인한 값, 표준에서 도출한 값, 추론한 값, 순수 합성값으로 나눈다. OPC UA 30200의 표준 Node와 단위는 표준 정의가 우선하고, 가상 제조사 성향은 추가 Node나 REST·MQTT 원천 필드에 적용한다. 이렇게 해야 테스트 데이터에 담긴 가정을 나중에 실제 제조사의 사양으로 오해하지 않는다. 원천 입력은 JSON Schema·XSD·CSV 열 스키마로, 변환 결과는 제조사 중립 스키마로 별도 검증하며 원본과 매핑 버전을 함께 보존한다.[6]

Mock 생성·제어의 주 구현 언어는 C#으로 결정됐다. C/C++은 C#·.NET을 사용할 수 없는 운영체제나 구형 장비·라이브러리 경계가 확인될 때에만 사용한다. 현재 결정은 언어 정책이지 Mock 프로그램 완성을 뜻하지 않는다.[3][8]

처리 단계	입력과 책임	보존 또는 판정
읽기 전용 수신	REST 조회·MQTT 구독·OPC UA 읽기 또는 파일·운영자 입력	원문, 소스·장비·배치와 수신시각 기록
순서 복원	장비별 순번을 짧은 버퍼에서 재정렬	5초 결손 대기 후 gap과 늦은 사건 분리
표준화	원천 필드·단위·코드를 공통 값 유형으로 변환	원본과 매핑 규칙 버전 병렬 보존
품질 판정	스키마·논리 범위·중복·누락을 검사	경고는 통과, 불가능한 값은 격리·검토

표 4 원천 이벤트가 조리 데이터가 되는 경로. 수집과 데이터 품질은 음식 자체의 합격 판정과 다르다.

예를 들어 vendor-b가 화씨 온도를 보냈을 때 섭씨로 바꾸는 것은 관계식이 정해진 변환이다. 그러나 rpm, 출력 %, 속도 level은 같은 숫자처럼 보여도 서로 다른 물리량이므로 관계식 없이 환산하지 않는다. 센서값의 결손을 통계값으로 자동 채우지도 않는다. 경고 데이터는 원인을 보며 사용할 수 있지만 물리적으로 불가능한 값은 정상 조리기록에서 격리한다.[6][11]

전달 실패도 데이터 의미에 영향을 준다. 장비별 순번이 비면 즉시 결손이라고 단정하지 않고 5초 동안 늦은 메시지를 기다린다. 그 안에 도착하면 순서를 복원하고, 만료되면 gap을 기록한 뒤 후속 메시지를 보낸다. 만료 후 도착한 원천 이벤트는 이미 작성된 판정을 덮어쓰지 않고 늦은 사건으로 남겨 새 파생 버전에서 재계산할 범위를 판단한다. 동일 업무 키에 내용까지 같은 재전송과, 같은 키인데 내용이 다른 충돌도 별도로 기록한다.[6][3]

품질검사는 오류를 모두 삭제하는 방식이 아니다. 통계적으로 이례적이지만 가능한 값은 경고와 검토 대상으로 보존하고, 스키마 위반이나 물리적으로 불가능한 값은 격리해 정상 분석과 학습 후보에서 제외한다. 자동 보정은 단위 변환이나 확정된 코드 매핑처럼 입력과 규칙만으로 같은 결과를 다시 만들 수 있는 경우로 제한한다. 구체적인 소프트·하드 한계와 지연·신선도 임계치는 대표 데이터와 시험 결과를 확보한 뒤 확정할 항목이다.[6]

5 조리 시나리오와 AI 권고

데이터 수집의 시험 대상은 개별 센서가 아니라 하나의 조리작업이다. 돈까스·튀김기, 제육볶음·볶음교반기, 고등어구이 정식·콤비오븐을 대표 합성 시나리오로 정했다. 온도·시간 외에 투입과 인양, 회전속도, 챔버 모드 전환, 운영자 관찰과 식사배치의 연결을 각각 확인하기 위한 조합이다. 이 메뉴의 합성 기준은 실제 식품 안전기준이나 제조사 조리법으로 일반화하지 않는다.[2]

돈까스 시나리오에서는 두께 프로파일과 배치중량을 고른 뒤 기름의 예열·투입 후 회복·인양 구간을 구분한다. 제육볶음에서는 고기·채소·소스의 투입사건과 목표 회전속도, 실제 모터 부하를 같은 작업 시간축에 놓는다. 고등어구이 정식에서는 밥의 스팀 조리와 생선의 건열 굽기를 서로 다른 작업으로 저장하되 하나의 식사배치로 연결한다. 모드 전환과 챔버 상태가 확인되지 않았다면 단순히 시간이 지났다는 이유로 다음 조리의 성공을 인정할 수 없다. 이 조건과 수치는 합성 시험 계약이며 실물 장비나 식품 안전 검증 결과는 아니다.[2]

조리작업의 완료, 공정 조건 통과, 식품 안전 판정, 사람의 품질평가는 서로 다른 결과다. 센서가 없어서 평가를 못한 상태를 '실패'나 0점으로 섞으면 원인을 잃는다. 품질평가는 재료 상태·전처리·맛·식감·외관·향·온도 적합성의 일곱 항목을 기록하도록 설계했다. 점수는 합성 평가척도의 제안 규칙이지 현장에서 달성한 품질 성적이 아니다.[2]

각 작업은 준비·예열, 투입·모니터링, 인양·평가, 권고 검토를 거치며 운영자 사건과 장비 시계열을 함께 사용한다. 작업 완료를 누른 시각과 실제 인양시각, 센서가 끊긴 상태와 측정했지만 기준에 미달한 상태를 구분한다. 안전 실패 뒤 추가 가열이 필요하면 원래 판정을 고치지 않고 새 공정 구간과 안전평가를 생성한다. 이 이력이 있어야 이후의 개선안이 어떤 작업과 관찰에서 나왔는지 되짚을 수 있다.[2]

AI는 매장 조건과 승인된 레시피, 사용 가능한 조리 이력을 받아 설명·추천·새 레시피 초안과 조건 변경 후보를 만든다. 수집 데이터가 불완전하거나 장비가 지원하지 않는 값 유형이면 답변을 보류하거나 추가 확인을 요구한다. 운영자는 변경 전후, 데이터 출처, 안전·품질 제약을 확인해 채택·수정·거부를 남긴다. AI 결과는 장비 제어 명령으로 전달되지 않는다.[2][7]

권고 후보가 승인된 레시피와 섞이지 않도록 메뉴, 레시피와 레시피 버전을 구분하고, 조리작업은 실제 사용한 버전 식별자를 고정한다. AI가 조건을 바꾸자고 제안해도 이전 버전은 그대로 보존되며, 권한 있는 사람이 검토·승인한 새 버전만 현장에 제공한다. 이렇게 해야 개선 전후 배치를 같은 기준으로 비교하고, 과거 작업이 어떤 조건으로 수행됐는지 재현할 수 있다. 세부 승인 상태와 동시 편집 계약은 아직 구현 전 상세화가 필요하다.[2][7]

기본 선택 모델은 로컬 Ollama의 llama3.1:8b이며 다른 로컬·외부 모델은 승인 상태, 기능별 등급, 비용 한도와 외부 전송 정책을 통과해야 한다. 모델 목록은 품질·지연 검증 결과가 아니다. 외부 키는 등록 서비스가 암호화해 저장하고 실제 호출은 AI Gateway만 맡는 구조를 제안했다. 공급자별 정확도와 성능은 구현 후 평가해야 한다.[3][8]

6 시스템 구조와 운영 안전장치

전체 흐름은 '가상 장비·파일·사람 입력 → 수집·순서 복원 → 표준화·품질 → 조리작업·평가 → AI 후보 → 운영자 승인'이다. 수집 어댑터, 업무 서비스, AI 실행과 화면을 논리적으로 나누어 원천 형식의 변화가 조리 판정 규칙을 직접 바꾸지 않게 한다. 저장 단계에서는 원본·표준값·판정·수정 이력을 연결하되 기존 기록을 덮어쓰지 않는다.[3][9]

REST Polling, MQTT 구독, OPC UA 읽기는 모두 플랫폼에서 장비 쪽으로 연결을 시작하는 수집 방식이다. 그래서 수집 진입점에 하나의 공통 인바운드 Load Balancer를 두는 설계가 아니라 프로토콜별 어댑터를 독립 모듈 또는 작업으로 분리한다. 소스·장비·배치별 순번을 복원한 이벤트는 장비별 순서를 유지하는 큐를 거쳐 표준화 작업자로 전달한다. 이 경계는 한 프로토콜의 연결 장애가 다른 장비의 조리 판정 규칙 자체를 바꾸지 않도록 하기 위한 것이다. 현재 ECS Fargate·DynamoDB·SQS FIFO는 설계 대상이며 실제 클라우드 적합성은 별도 검증이 필요하다.[3]

논리 경계	핵심 책임	현재 증거 수준
수집과 큐	외부로 시작하는 읽기 연결, 순번 복원, 장비별 순서 유지	SAD·UML 설계
업무와 저장	조리작업, 버전형 평가, 데이터 의미·접근 패턴	SRS·ERD·테이블·코드북 설계
AI와 사람	정책 검사·공급자 호출·후보 생성·검토·승인	SAD·IA·UML 설계
보호와 운영	목적별 접근, 감사, 삭제, 백업·복원	정책과 설계, 실제 AWS 시험 전

표 5 프로그램의 논리 경계. 표의 서비스는 배포 개수나 실행 완료 상태를 나타내지 않는다.

고객 화면은 자신의 매장 작업·레시피·대화만, 관리자 화면은 설정과 감사 등 별도 권한을 다룬다. 화면 메뉴를 숨기는 것만으로 권한을 보장하지 않으므로 업무 API에서도 사용자·역할·매장 범위를 다시 검사한다. 오래된 값, 미수신, 일부 도착, 권한 거부는 서로 다른 상태로 표시해야 한다.[7]

프로그램 내부에서는 화면·HTTP 계층이 요청을 받고, application 계층이 사용 사례의 처리 순서를 조정하며, domain 계층이 조리 상태와 평가 규칙을 소유하도록 제안했다. 저장소, 메시지 큐와 AI 공급자 SDK는 바깥쪽 인프라 경계에 둔다. 이렇게 나누면 vendor-b의 필드명이 바뀌거나 AI 모델을 교체할 때 원천 어댑터와 공급자 어댑터를 수정하고, 이미 정의된 조리 판정의 의미는 별도로 지킬 수 있다. 이것은 구현 방향이며 실제 저장소·패키지 구조와 배포 개수는 아직 확정되지 않았다.[3][8]

AI 호출에는 또 다른 신뢰 경계가 있다. 업무 큐에는 작업 ID와 만료 정보만 싣고, 요청 본문은 별도 암호화 저장소에 두는 Claim Check 방식을 제안한다. 모델의 승인·활성 상태, 기능 등급, 비용 한도와 외부 전송 정책을 검사한 뒤

Gateway가 공급자 키를 사용한다. 일반 업무 서비스나 화면이 키를 직접 복호화하도록 만들면 승인·감사의 단일 경로를 잃는다. 이 설계의 권한 정책과 장애 복귀는 실제 환경에서 시험해야 한다.[3]

개인정보 원문은 별도 Vault에 두고 업무 데이터에는 목적에 맞는 가명값을 쓰는 설계다. 삭제 요청은 저장소별 제거와 복원 후 재삭제 확인이 끝나야 완료로 기록한다. 감사 이력은 원문 프롬프트와 저장소·키·권한을 분리한다. 이러한 경계는 설계 원칙이지 현재 실제 AWS 권한 정책이 검증됐다는 뜻이 아니다.[3][10]

7 현재 검증 결과와 해석 범위

2026년 10월 23일의 4.2 문서 정합성 검토에서는 당시 승인된 정책 사이의 직접 충돌을 발견하지 못했다. 이어 Python 표준 라이브러리와 메모리 SQLite로 매핑 입력, 중복·원본 불변, 평가 이력과 결측, 삭제 후 복원 재삭제의 작은 계약 예시를 실행했다. 검증 기록은 18개 로컬 점검 통과와 별도 후보 5개 점검 통과를 적고 있다. 이 숫자는 작은 예제의 검사 수이며 전체 요구사항 완료율이나 서비스 신뢰도 점수가 아니다.[4]

18개 점검은 후보 직렬화에서 필드 순서와 수치 표기가 결정적인지, 허용되지 않은 매핑 연산자를 거부하는지, 같은 업무키의 재전송과 내용 충돌을 원본을 지우지 않고 구별하는지 등을 살폈다. 평가 수정에서는 최초 평가를 남기고 분모가 없는 미산출과 유효한 0을 구분했으며, 삭제 예시에서는 남은 데이터가 있으면 완료 처리하지 않고 오래된 백업에서 돌아온 값을 다시 삭제하도록 했다. 모두 축소된 입력과 메모리 저장소에서 확인한 계약의 예이며 실제 사용자 화면이나 분산 저장소에서 재현한 결과는 아니다.[4]

후보 비교에서 길이 접두 HMAC 입력, 결정적 JSON 직렬화 방향, 평가의 append-only 버전과 현재 포인터, DynamoDB On-demand와 감사 단일 Writer를 설계 초안으로 승인했다. 특히 DynamoDB 용량과 감사 Writer는 로컬 스크립트로 측정하거나 검증하지 않았다. AWS KMS·STS·SQS·DynamoDB, 분산 동시성, 실제 Mock 프로토콜, AI 라이브러리와 사용자 화면은 검증 범위 밖이다.[4]

별도 후보 5개 점검은 길이 접두 방식이 단순 구분자 결합에서 생길 수 있는 바이트 충돌을 피하는지, Python과 JavaScript의 기본 JSON 출력 차이가 내용 해시에 어떤 영향을 주는지, 오래된 기대 버전으로 평가를 동시에 바꾸려는 시도를 막을 수 있는지에 초점을 두었다. 여기서 승인된 것은 후속 구현의 출발점이지 암호화 키, 해시 알고리즘, DynamoDB 인덱스나 감사 처리량의 최종 선택이 아니다. 실제 교차 언어 바이트 일치와 저장소 조건부 쓰기·재시도 동작을 확인해야 한다.[4]

확인한 것	아직 확인하지 않은 것	다음 검증 환경
일부 규칙의 결정성·원본 불변·결측 구분	실제 저장소의 키·인덱스·경쟁 처리	초기 구현 후 실제 데이터 저장소
가상 입력에 대한 로컬 계약 예시	REST·MQTT·OPC UA 송수신과 장비별 오류 복귀	C# Mock 및 통합 어댑터
정책과 설계 문서 간 주요 경계	KMS·STS 권한, 삭제·복원, 처리량·비용	AWS 계정과 격리 클라우드 시험

표 6 검증 결과의 적용 범위. 로컬 계약 통과는 실제 서비스 적합 판정을 대신하지 않는다.

검증 기록은 작성 시점의 이전 산출물 버전도 언급한다. 보고일의 공개 최신본은 RD·SRS 0.7, DCD 0.6, SAD 0.3 등으로 바뀌었으므로, 과거 점검을 새 문서와 물리 설계 전체의 재검증 결과로 간주하지 않는다. 새 구현 계약이 정해질 때 같은 경계를 다시 시험해야 한다.[3][4]

검증 순서도 목적에 따라 나눈다. 우선 최신 문서와 실행 입력의 필드·상태·예외가 일치하는지 확인한다. 다음으로 C# Mock과 어댑터를 연결해 정상·중복·결손·재연결 사건의 원본, 매핑, 품질 판정을 추적한다. 마지막으로 AWS 환경에서 tenant 권한, 큐와 저장소의 경쟁 처리, 삭제·복원 및 비용·처리량을 측정한다. 앞 단계의 통과가 뒤 단계의 통과를 보장하지는 않으므로 각 환경의 증적을 따로 남겨야 한다.[3][4]

8 남은 결정과 다음 단계

첫째, 구현 착수 전에는 Mock의 실제 payload·연계 계약, 매핑 규칙 스키마와 실행 한도, 장비·메뉴별 화면·시퀀스, 평가 포인터와 저장 키의 동시성 계약을 확정해야 한다. 현재 문서가 `TBD` 또는 후속 검토로 남긴 부분이다. 개발자는 임의로 빈칸을 채우기보다 결정과 변경 영향을 기록한 뒤 산출물을 갱신해야 한다.[6][3]

이때 모든 제조사 형식을 한꺼번에 구현할 필요는 없다. 첫 수직 흐름에서는 하나의 fixture가 보낸 정상 이벤트와 오류 이벤트를 같은 조리작업에 연결하고, 원본·표준값·품질 finding·공정판정이 각각 어떤 ID와 버전으로 남는지 확인하는 편이 유용하다. 이후 연계 방식과 fixture를 늘리며 같은 업무 규칙이 유지되는지 회귀시험한다. 적용할 첫 프로파일, 시험 입력의 개수와 통과 기준은 구현·테스트 계획에서 명시해야 하며 현재 성과로 계산하지 않는다.[2][6][3]

둘째, C# Mock 제어 프로그램과 읽기 전용 수집 경로부터 구현하고, 정상·오류·재연결·순번 결손의 합성 사건을 원본 보존·표준화·품질 판정까지 연결해야 한다. 그다음 조리작업, 사람의 평가, AI 후보 생성·검토를 같은 작업 ID로 이어야 한다. 이 순서는 데이터와 승인 근거가 없는 AI 화면을 먼저 완성한 것으로 오인하지 않기 위한 제안이다.[2][3]

셋째, 실제 AWS 환경에서는 KMS·STS tenant 경계, 큐·저장소 동시성, 삭제 후 복원 재삭제, 감사 연속성을 시험한다. 대표 데이터와 부하 결과가 생긴 뒤 품질·성능 임계치, 최소 표본, 운영 알림과 복원 목표를 확정한다. 공개 데모 전에는 보안·권한·오류 복귀와 운영 절차를 함께 검증해야 한다.[3][4]

다음 중간 점검에서는 작업별로 입력 fixture와 시드, 적용한 스키마·매핑 버전, 예상 상태와 실제 결과, 실행 환경을 나란히 제시할 필요가 있다. 그래야 '설계상 가능한 흐름'과 '재현된 기능'을 구별할 수 있다. 특히 AI 권고는 답변의 유창성만으로 판정하지 않고, 사용한 데이터의 완전성, 제약 위반 여부, 사람의 승인 기록과 사용 레시피 버전까지 함께 확인해야 한다.[2][6][3]

중간 시점의 성과는 사업 요구를 추적 가능한 개인 프로젝트 범위로 좁히고, 개발자와 테스터가 읽을 최신 분석·설계 문서를 마련한 데 있다. 남은 가장 큰 위험은 문서의 논리 경계가 실제 구현과 클라우드 권한·저장소에서 같은 방식으로 성립하는지 아직 증명하지 못했다는 점이다. 따라서 다음 보고에서는 문서 수보다 실행 가능한 흐름과 재현 가능한 시험 증적을 중심으로 진척을 판정해야 한다.

참고문헌

[1] 요구사항정의서(RD)_v0.7.pdf, 2026년 10월 23일. 자료실 원본

[2] 요구사항명세서(SRS)_v0.7.pdf, 2026년 10월 23일. 자료실 원본

[3] 시스템아키텍처정의서(SAD)_v0.3.pdf, 2026년 10월 23일. 자료실 원본

[4] 4.2·4.3 부분 검증 기록 v0.3, 2026년 10월 23일, 원 프로젝트 validation 디렉터리.

[5] 과업지시서.pdf, 2026년 8월. 자료실 원본

[6] 데이터수집정의서(DCD)_v0.6.xlsx, 2026년 10월 23일. 자료실 원본

[7] 스토리보드(Storyboard)_v0.4.pdf, 2026년 10월 23일. 자료실 원본

[8] 개발표준정의서_v0.4.pdf, 2026년 10월 23일. 자료실 원본

[9] 논리ERD_v0.2.pdf 및 시퀀스-클래스다이어그램_v0.3.pdf, 2026년 10월 23일. 자료실 원본 UML 열람본

[10] 테이블정의서_v0.2.xlsx, 2026년 10월 23일. 자료실 원본

[11] 코드정의서(Codebook)_v0.2.xlsx, 2026년 10월 23일. 자료실 원본